SQL SELECT Statements

topics: SELECT, WHERE, ORDER BY, DISTINCT, aggregate functions, GROUP BY, HAVING, date functions, string functions, numeric functions and CASE.

Sample Table: STUDENT

ID	NAME	DEPT	MARKS	AGE	CITY
1	Arun	CS	85	22	Kochi
2	Anu	CS	72	21	Kochi
3	Rahul	MCA	91	23	Trivandrum
4	Meera	CS	65	22	Kollam
5	John	MCA	78	24	Kochi
6	Diya	BCA	88	21	Kottayam

1. Basic SELECT

SELECT chooses the columns you want to display.
All columns:

```
SELECT * FROM STUDENT;
```

Selected columns:

```
SELECT NAME, MARKS, CITY FROM STUDENT;
```

Tip: Remember: SELECT = what you want to see; FROM = where the data comes from.

2. WHERE — Filtering Rows

WHERE filters individual rows according to a condition.

```
SELECT * FROM STUDENT WHERE MARKS > 80;
```

Common operators: =, >, <, >=, <=, <>.
Strings use single quotes:

```
SELECT * FROM STUDENT WHERE DEPT = 'CS';
```

Tip: AND requires both conditions. OR requires at least one condition.

3. DISTINCT

DISTINCT removes duplicate values from the result.

```
SELECT DISTINCT CITY FROM STUDENT;
```

Useful when the question asks for unique or different values.

4. ORDER BY

ORDER BY sorts the result. ASC means ascending; DESC means descending.

```
SELECT NAME, MARKS FROM STUDENT ORDER BY MARKS DESC;
```

Multiple-column sorting:

```
SELECT * FROM STUDENT ORDER BY DEPT ASC, MARKS DESC;
```

Tip: ASC is the default sorting direction.

5. Aggregate Functions

Aggregate functions calculate a value from multiple rows.

```
SELECT COUNT(*) FROM STUDENT;
SELECT SUM(MARKS) FROM STUDENT;
SELECT AVG(MARKS) FROM STUDENT;
SELECT MAX(MARKS) FROM STUDENT;
SELECT MIN(MARKS) FROM STUDENT;
```

Memory trick:

```
COUNT = how many? SUM = total? AVG = average? MAX = highest? MIN = lowest?
```

6. GROUP BY

GROUP BY creates groups so an aggregate function can be calculated for each group.

```
SELECT DEPT, AVG(MARKS)
FROM STUDENT
GROUP BY DEPT;
```

Number of students in each department:

```
SELECT DEPT, COUNT(*)
FROM STUDENT
GROUP BY DEPT;
```

Tip: Think: first make groups, then calculate within each group.

7. HAVING

HAVING filters groups after GROUP BY. WHERE filters rows before grouping.

```
SELECT DEPT, COUNT(*)
FROM STUDENT
GROUP BY DEPT
HAVING COUNT(*) > 2;
```

Example with average:

```
SELECT DEPT, AVG(MARKS)
FROM STUDENT
GROUP BY DEPT
HAVING AVG(MARKS) > 75;
```

Tip: Memory trick: WHERE → rows; HAVING → groups.

8. WHERE + GROUP BY + HAVING

These can be combined when you need to filter rows, form groups, and then filter the groups.

```
SELECT DEPT, AVG(MARKS)
FROM STUDENT
WHERE MARKS > 60
GROUP BY DEPT
HAVING AVG(MARKS) > 75;
```

Logical processing order:

```
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
```

9. String Functions

Common Oracle string functions include UPPER, LOWER, LENGTH and SUBSTR. The || operator concatenates strings.

```
SELECT UPPER(NAME) FROM STUDENT;
SELECT LOWER(NAME) FROM STUDENT;
SELECT NAME, LENGTH(NAME) FROM STUDENT;
SELECT SUBSTR(NAME, 1, 3) FROM STUDENT;
SELECT NAME || ' - ' || CITY FROM STUDENT;
```

These are useful for questions involving names, text formatting and character counts.

10. Numeric Functions

Common Oracle numeric functions include ROUND, CEIL, FLOOR and MOD.

```
SELECT ROUND(85.678, 2) FROM DUAL;
SELECT CEIL(85.2) FROM DUAL;
SELECT FLOOR(85.9) FROM DUAL;
SELECT MOD(10, 3) FROM DUAL;
```

ROUND rounds to a chosen number of decimal places; CEIL rounds upward; FLOOR rounds downward; MOD returns the remainder.

11. CASE — IF/ELSE in SQL

CASE allows conditional output.

```
SELECT NAME, MARKS,
CASE
WHEN MARKS >= 80 THEN 'Excellent'
WHEN MARKS >= 60 THEN 'Good'
ELSE 'Needs Improvement'
END AS PERFORMANCE
FROM STUDENT;
```

Think of CASE as an SQL version of IF / ELSE IF / ELSE.

12. Date Functions

For Oracle date columns, SYSDATE returns the current database date/time. EXTRACT can obtain parts such as year, month and day.

```
SELECT SYSDATE FROM DUAL;
SELECT EXTRACT(YEAR FROM JOIN_DATE) FROM STUDENT;
SELECT EXTRACT(MONTH FROM JOIN_DATE) FROM STUDENT;
SELECT EXTRACT(DAY FROM JOIN_DATE) FROM STUDENT;
```

Use date functions when a question asks about years, months, days or the current date.

Most Important Exam Pattern

A SELECT query can follow this general structure. You do not need every clause in every query.

```
SELECT column(s)
FROM table
WHERE condition
GROUP BY column(s)
HAVING group_condition
ORDER BY column(s);
```

WHERE vs HAVING

WHERE	HAVING
Filters individual rows	Filters groups
Normally used before GROUP BY	Used after GROUP BY
Example: MARKS > 80	Example: COUNT(*) > 2

Practice Questions

1. Display all columns from STUDENT.
2. Display only NAME and MARKS.
3. Display students whose MARKS are greater than 80.
4. Display unique CITY values.
5. Display students ordered by MARKS from highest to lowest.
6. Find the total number of students.
7. Find the highest and lowest MARKS.
8. Find the average MARKS for each DEPT.
9. Find the number of students in each DEPT.
10. Display only departments having more than 2 students.
11. Display NAME in uppercase.
12. Display NAME and the first 3 characters of NAME.
13. Classify students as Excellent (80+), Good (60–79), or Needs Improvement (below 60) using CASE.

Quick Revision Sheet

SELECT → choose columns

FROM → choose table

WHERE → filter rows

DISTINCT → remove duplicates

ORDER BY → sort results

COUNT/SUM/AVG/MAX/MIN → calculate summaries

GROUP BY → create groups

HAVING → filter groups

UPPER/LOWER/LENGTH/SUBSTR → work with strings

ROUND/CEIL/FLOOR/MOD → numeric operations

CASE → conditional output

SYSDATE/EXTRACT → work with dates